

# Delivery Issue Report Handling

Use Case Competition – Chat Export & Solution Strategy

## 1. Use Case Understanding

The organization receives a continuous stream of customer reports about missing, late, or damaged deliveries. Some reports request a refund, credit, account change, or replacement shipment. The application must interpret each request, verify the claim against delivery data, apply business rules, execute eligible resolutions autonomously, support human review/override, and persist the complete decision and action history.

- Ingest `delivery_issue_reports.csv`, `delivery_tracking_records.csv`, and `replacement_shipment_history.csv`.
- Use the in-house LLM to interpret the customer's actual request and identify the delivery concerned.
- Operate as an agent using tools to retrieve delivery evidence and validate the request.
- Only auto-resolve requests below a reasonable value threshold.
- Only auto-resolve when the customer's stated reason is supported by the delivery-tracking record.
- Execute refund, credit, account-change, or replacement actions through tools.
- For replacement requests, check recent replacement history and the applicable cap/window before scheduling.
- Show the decision/evidence and autonomous actions, while allowing human review and override.
- Persist requests, decisions, actions, scheduled replacements, and overrides.

## 2. Dataset Understanding

### Dataset 1 – `delivery_issue_reports.csv`

| Column                     | Purpose                                                |
|----------------------------|--------------------------------------------------------|
| <code>request_id</code>    | Unique customer issue/request identifier               |
| <code>customer_name</code> | Customer associated with the request                   |
| <code>customer_tier</code> | Customer service tier                                  |
| <code>tracking_id</code>   | Delivery identifier used to join with tracking records |
| <code>submitted_at</code>  | Request submission timestamp                           |
| <code>request_text</code>  | Natural-language issue and requested resolution        |

### Dataset 2 – `delivery_tracking_records.csv`

| Column                         | Purpose                                  |
|--------------------------------|------------------------------------------|
| <code>tracking_id</code>       | Unique delivery identifier               |
| <code>customer_name</code>     | Customer associated with delivery        |
| <code>item_description</code>  | Product shipped                          |
| <code>order_value</code>       | Monetary value of the order              |
| <code>verified_category</code> | Operationally verified delivery issue    |
| <code>credit_eligible</code>   | Whether credit is permitted              |
| <code>credit_value</code>      | Credit amount associated with the record |

Dataset 3 – replacement\_shipment\_history.csv

| Column                             | Purpose                                             |
|------------------------------------|-----------------------------------------------------|
| customer_name                      | Customer identifier                                 |
| replacement_shipments_last_30_days | Number of replacement shipments in the last 30 days |

3. Core Data Relationships

delivery\_issue\_reports.tracking\_id → delivery\_tracking\_records.tracking\_id. For replacement requests, customer\_name → replacement\_shipment\_history.customer\_name.

The key distinction is: the LLM interprets what the customer says; the tracking system determines what actually happened; the policy engine determines whether the requested action is permitted.

4. Recommended Solution Philosophy

Evidence-backed autonomous resolution of delivery disputes under controlled business policies.

The recommended architecture separates four responsibilities: LLM interpretation, tool-based evidence retrieval, deterministic policy evaluation, and tool-based action execution. This prevents an LLM from directly controlling financial or eligibility decisions.

5. Candidate Approaches

**A. Pure LLM Agent:** Simple and demo-friendly, but unsafe because the model could resolve a request despite contradictory tracking evidence.

**B. Traditional Rule Engine:** Deterministic and inexpensive, but brittle for natural-language customer requests.

**C. Hybrid LLM + Deterministic Policy Engine:** Recommended. The LLM handles language ambiguity, while code controls evidence validation, thresholds, eligibility, and actions.

6. Recommended Agent Workflow

Customer Request → LLM Intent Parser → Agent Orchestrator → Delivery/Replacement Tools → Evidence Validator → Policy Engine → Auto Resolution or Human Review → Action Tools → Audit Database/Dashboard.

7. Evidence-First Principle

**No evidence, no autonomous action.** Every automated resolution should have an evidence chain: request → interpretation → retrieved record → evidence match → policy evaluation → action → audit entry.

8. Resolution Confidence

Confidence should represent operational evidence rather than merely the LLM's self-reported confidence. Useful factors include intent confidence, tracking match, reason verification, value-threshold eligibility, and replacement eligibility. Confidence must never override a contradictory policy or evidence result.

9. Resolution State Machine

RECEIVED → INTERPRETED → EVIDENCE\_FETCHED → VALIDATED → POLICY\_EVALUATED → AUTO\_RESOLVE / HUMAN\_REVIEW → ACTIONED → CLOSED.

10. Replacement Mini-Agent

Replacement requested → verify delivery issue → check eligibility → retrieve 30-day replacement history → compare against configured cap → schedule replacement if eligible; otherwise route to human review.

## 11. Human Review Console

The operator should see the customer request, interpreted intent, tracking evidence, policy checks, proposed action, and action status. The interface should provide explicit Approve and Override actions.

## 12. Explainability

Expose an audit explanation rather than hidden model chain-of-thought. For example: Decision, Evidence, Eligibility, and Action. For blocked requests, explain exactly which evidence or policy condition prevented automation.

## 13. Economic Architecture

For a 24-hour prototype, avoid unnecessary infrastructure such as Kubernetes, Kafka, Redis, vector databases, and microservices. A practical stack is Python + FastAPI + agent orchestrator + in-house LLM + SQLite/PostgreSQL + CSV ingestion, with Streamlit or a lightweight React UI.

LLM calls should be minimized. Do not send the entire database to the model. Extract structured intent first, then retrieve only the relevant delivery/customer records through tools.

## 14. Policy Configuration

Keep business rules configurable rather than scattering hard-coded conditions throughout the application. The exact refund threshold and replacement cap should be established from the competition's supplied data/rules, not invented arbitrarily.

## 15. Idempotency and Action Ledger

Financial or shipment actions should use an idempotency key such as request\_id + action\_type so retries cannot create duplicate refunds or replacements. An action ledger should retain action\_id, request\_id, action\_type, amount, timestamp, decision source, execution status, human override, and idempotency key.

## 16. Dry-Run Mode

Before autonomous actions are enabled, process the dataset in dry-run mode and display proposed resolutions. This provides a safe validation stage before enabling live execution.

## 17. Batch Processing and Dashboard

Because the brief describes a continuous stream, support both single-request and batch processing. A dashboard can show total requests, auto-resolved cases, human-review cases, rejected cases, credits issued, and replacements scheduled.

## 18. Agent Trace

Show operational events such as request received, intent extracted, delivery lookup, evidence validated, policy evaluated, action executed, and audit persisted.

## 19. Strongest Differentiators

- **Evidence before action:** No supported evidence means no autonomous resolution.
- **LLM for ambiguity, code for policy:** Natural language is handled by AI; financial and eligibility rules remain deterministic.
- **Progressive autonomy:** Strongly supported low-risk cases are automated; ambiguous, contradictory, or high-value cases go to humans.
- **Auditable decisions:** Every resolution follows Request → Evidence → Policy → Decision → Action.
- **Cost-conscious architecture:** Minimal LLM calls, targeted tool retrieval, and lightweight infrastructure.

## 20. Recommended Competition MVP

Implement the core spine: CSV ingestion → LLM parser → agent orchestrator → delivery/replacement tools → evidence check → policy engine → auto action/human review → audit database → dashboard.

Prioritize three differentiators in the demo: (1) Resolution Evidence Chain, (2) Progressive Autonomy, and (3) Dry-Run/Policy Simulation.

## 21. Important Design Constraint

Do not invent the refund threshold or replacement cap. The actual CSV files and competition rules should be inspected before implementation so that the final decision engine reflects the intended business logic.